

Regras Lógicas e Simplificação Booleana – NCAS

Responsável: Maria (Engenharia de Lógica e Arquitetura)

Etapa 1 – Cenários Operacionais

Antes de sair programando, pensei em algumas situações que podem acontecer na colônia e que dariam pra virar regra no sistema. Escolhi 4 cenários, um pra cada tipo de dado que a gente vai trabalhar (acesso, alertas, solicitações e segurança). Nessa etapa só escrevi a ideia e a expressão booleana do jeito que ela sai da cabeça, sem simplificar nada.

Cenário 1 – Liberação de consulta ao sistema

Nem todo mundo pode ver os dados de qualquer módulo. Só faz sentido liberar a consulta se a pessoa tiver autorização e o módulo estiver ativo.

Variáveis:

AUTORIZADO → usuário tem permissão de acesso

MODULO_ATIVO → módulo está funcionando normalmente

CONSULTA_LIBERADA = AUTORIZADO AND MODULO_ATIVO

Cenário 2 – Geração de alerta operacional

O sistema precisa avisar quando algo está errado. Pode ser uma falha crítica no módulo ou o consumo de energia passando do limite seguro. Uma coisa já é suficiente pra disparar alerta.

Variáveis:

FALHA_CRITICA → módulo apresentou falha grave

CONSUMO_ELEVADO → consumo acima do limite estabelecido

GERAR_ALERTA = FALHA_CRITICA OR CONSUMO_ELEVADO

Cenário 3 – Priorização de solicitações da tripulação

Uma solicitação só vira prioridade máxima se for marcada como urgente e vier de um setor essencial, tipo suporte de vida ou navegação.

Variáveis:

URGENTE → solicitação marcada como urgente

SETOR_ESSENCIAL → solicitação vem de setor essencial da colônia

PRIORIDADE_MAXIMA = URGENTE AND SETOR_ESSENCIAL

Cenário 4 – Bloqueio de operação por segurança

Tem duas coisas que, sozinhas, já travam uma operação: falha de segurança (tentativa de acesso indevido) ou inconsistência nos dados (dado corrompido, incompleto, fora do padrão). Se qualquer uma aparecer, bloqueia até alguém revisar.

Variáveis:

FALHA_SEGURANCA → houve problema de segurança detectado

INCONSISTENCIA_DADOS → dados não batem com o esperado

BLOQUEAR_OPERACAO = FALHA_SEGURANCA OR INCONSISTENCIA_DADOS

Etapa 2 – Simplificação com De Morgan (Cenário 4)

Escolhi o Cenário 4 pra aplicar o teorema porque dá pra montar ele de um jeito mais complexo primeiro e depois simplificar até chegar na expressão original.

Ponto de partida:

A operação só continua normal se não houver falha de segurança e não houver inconsistência nos dados.

$$\text{OPERACAO_NORMAL} = \text{NOT}(\text{FALHA_SEGURANCA}) \text{ AND } \text{NOT}(\text{INCONSISTENCIA_DADOS})$$

O bloqueio é o contrário disso — é quando a operação não está normal:

$$\text{BLOQUEAR_OPERACAO} = \text{NOT}(\text{OPERACAO_NORMAL})$$
$$\text{BLOQUEAR_OPERACAO} = \text{NOT}(\text{NOT}(\text{FALHA_SEGURANCA}) \text{ AND } \text{NOT}(\text{INCONSISTENCIA_DADOS}))$$

Aplicando De Morgan:

A negação de um AND vira um OR das negações — $\text{NOT}(A \text{ AND } B) = \text{NOT}(A) \text{ OR } \text{NOT}(B)$. Aqui A e B já são negações, então elas se cancelam:

$$\text{BLOQUEAR_OPERACAO} = \text{NOT}(\text{NOT}(\text{FALHA_SEGURANCA})) \text{ OR } \text{NOT}(\text{NOT}(\text{INCONSISTENCIA_DADOS}))$$
$$\text{BLOQUEAR_OPERACAO} = \text{FALHA_SEGURANCA} \text{ OR } \text{INCONSISTENCIA_DADOS}$$

Por que continua valendo o mesmo resultado:

Dizer "a operação não está normal" é logicamente igual a dizer "ou teve falha de segurança, ou teve inconsistência (ou os dois)". De Morgan só troca a forma de escrever — AND negado vira OR — mas o resultado lógico é idêntico. Dá pra confirmar isso batendo tabela-verdade: nos dois lados, o resultado só é falso quando as duas variáveis são falsas ao mesmo tempo.

Essa expressão final é a mesma que eu já tinha escrito "de cabeça" na Etapa 1 — a diferença é que agora eu cheguei nela partindo de uma forma mais complexa e aplicando o teorema, em vez de só escrever direto.

Próximo passo:

Na Etapa 3, essa regra simplificada vira uma condicional dentro do código Python (if FALHA_SEGURANCA or INCONSISTENCIA_DADOS:).